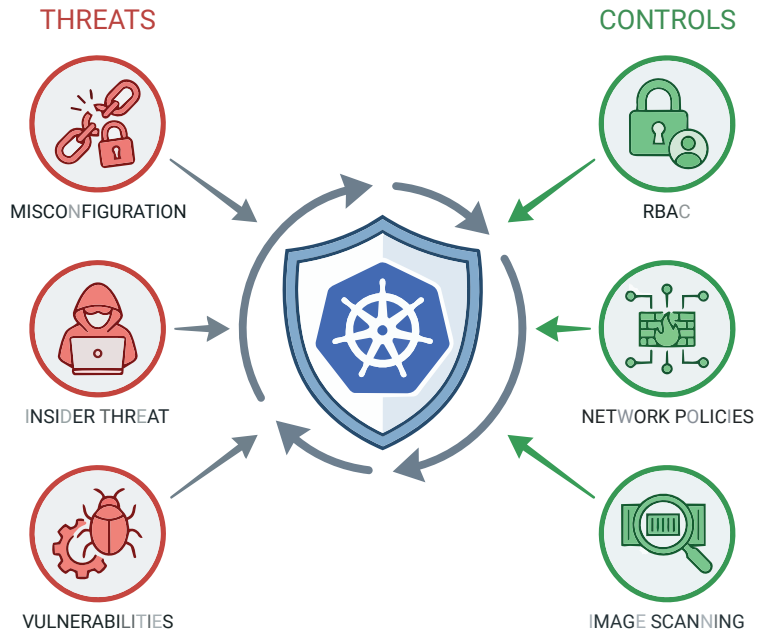


Kubernetes Security: Threats and Controls

Kubernetes has become the *de facto* standard for orchestrating containerised workloads at scale, but its distributed architecture introduces a broad attack surface spanning the control plane, worker nodes, network, and the software supply chain that builds container images. Thankfully, there are a number of controls that can help to secure systems.



Kubernetes (K8s) automates the deployment, scaling, and management of containerised applications. Its power comes from a highly programmable API server that mediates every action inside the cluster — from creating a pod to modifying network rules. This same programmability, however, means that a single misconfiguration (an overly permissive role, an open network path, an unscanned image) can expose the entire cluster to compromise. Industry data cited in recent reports shows that most organisations now run Kubernetes in hybrid-cloud environments and are increasing their reliance on it as a primary application platform, which raises the stakes for getting security right from the outset.

Kubernetes architecture overview

A Kubernetes cluster is divided into

a control plane, which makes global decisions about the cluster (scheduling, detecting/responding to events), and worker nodes, which run the actual application workloads inside pods (Figure 1). The control plane's kube-apiserver is the single-entry point for all requests and is therefore the most security-critical component: every authentication, authorisation Role-based access control (RBAC), and admission-control check happens here before a request is persisted to etcd, the cluster's key-value store of records.

Why Kubernetes security matters

The risks described in this article are not theoretical. In 2018, security researchers at RedLock discovered that Tesla's Kubernetes Dashboard had been left exposed to the public internet with no authentication at all (Figure 2). Attackers found the console, used it

to obtain AWS credentials embedded in a pod's environment, and deployed cryptocurrency-mining containers across Tesla's cloud infrastructure — deliberately throttling CPU usage to stay under the radar of monitoring systems. The incident illustrates, in one real example, how a single missing control (authentication on a management interface) cascades into credential exposure, unauthorised workload deployment, and sustained resource theft. We will revisit this and two further real-world cases in detail later.

Core controls of Kubernetes security

Role-based access control (RBAC):

RBAC governs who (a user, group, or ServiceAccount) can perform which verbs (get, list, create, delete, etc) on which resources (pods, secrets, deployments) in the cluster. A Role or ClusterRole defines a set of permissions;